EXPT NO: 9	CPM - ANALYSIS
DATE:	

AIM:

To perform Critical Path Method (CPM) Analysis for project scheduling and management using Python.

PROCEDURE:

1. Install Required Libraries:
o Make sure you have pandas, networkx, and matplotlib installed. You can install them using pip if not already installed.
bash
Copy code
pip install pandas networkx matplotlib

2. Define the Problem:
o Identify the tasks, their durations, and dependencies.

3. Set Up the Problem in Python:
o Use NetworkX to represent the project tasks and their dependencies.
o Use the topological sort to identify the critical path.

4. Run the Code:
o Execute the Python code to get the critical path and project duration.

5. Analyze the Output:
o Interpret the results and validate the solution against the project schedule.

PROGRAM:

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
```

```
# Define tasks
tasks = {
    'A': {'duration': 3, 'dependencies': []},
    'B': {'duration': 2, 'dependencies': ['A']},
    'C': {'duration': 4, 'dependencies': ['A']},
    'D': {'duration': 2, 'dependencies': ['B']},
    'E': {'duration': 3, 'dependencies': ['C']},
    'F': {'duration': 1, 'dependencies': ['D', 'E']}
}

# Create graph
G = nx.DiGraph()

# Add nodes and edges
for task, data in tasks.items():
    G.add_node(task, duration=data['duration'])
    for dep in data['dependencies']:
        G.add_edge(dep, task)

# Topological order
topological_order = list(nx.topological_sort(G))

earliest_start = {task: 0 for task in topological_order}
earliest_finish = {task: 0 for task in topological_order}

for task in topological_order:
    earliest_finish[task] = earliest_start[task] + G.nodes[task]['duration']
    for successor in G.successors(task):
        earliest_start[successor] = max(
            earliest_start[successor],
            earliest_finish[task]
        )

latest_finish = {task: earliest_finish[task] for task in topological_order}
latest_start = {task: earliest_start[task] for task in topological_order}


for task in reversed(topological_order):
    for predecessor in G.predecessors(task):
        latest_finish[predecessor] = min(
            latest_finish[predecessor],
            latest_start[task]
        )
        latest_start[predecessor] = (
```

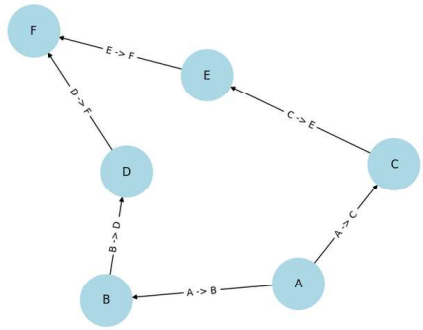
```
            latest_finish[predecessor] - G.nodes[predecessor]['duration']
        )
critical_path = [
    task for task in topological_order
    if earliest_start[task] == latest_start[task]
]

print("Earliest Start Times:", earliest_start)
print("Earliest Finish Times:", earliest_finish)
print("Latest Start Times:", latest_start)
print("Latest Finish Times:", latest_finish)
print("Critical Path:", critical_path)

pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_size=3000, node_color='lightblue')
nx.draw_networkx_edge_labels(
    G, pos,
    edge_labels={(u, v): f"{u}->{v}" for u, v in G.edges}
)

plt.show()
```

OUTPUT:

Earliest Start Times: {'A': 0, 'B': 3, 'C': 3, 'D': 5, 'E': 7, 'F': 10}
Earliest Finish Times: {'A': 3, 'B': 5, 'C': 7, 'D': 7, 'E': 10, 'F': 11}

Latest Start Times: {'A': 0, 'B': 5, 'C': 3, 'D': 7, 'E': 7, 'F': 10}
Latest Finish Times: {'A': 3, 'B': 7, 'C': 7, 'D': 9, 'E': 10, 'F': 11}

Critical Path: ['A', 'C', 'E', 'F']



RESULT:
Thus, solving CPM has been completed successfully.